

Advanced Natural Language Processing

Exploring Multi-Agent Systems with CrewAI

Sher Muhammad Daudpota
Sukkur IBA University

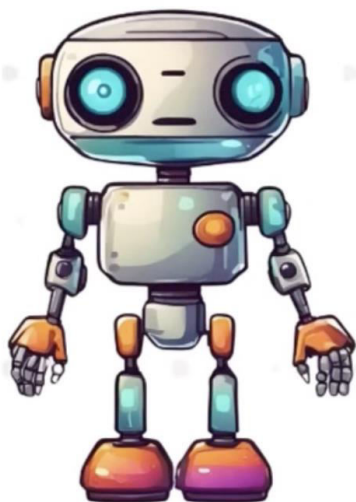
What is CrewAI?

CrewAI is an Agent Framework that allows us to create agents to help us solve simple or complex tasks.

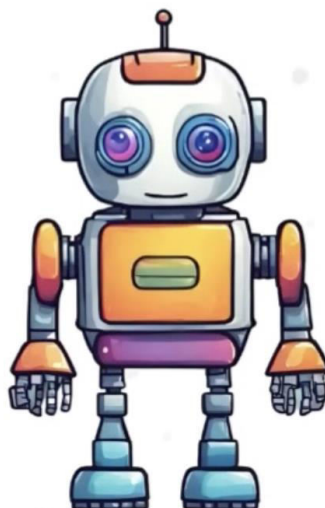
AI agents assume **roles**, share **goals**, and operate in a cohesive unit – much like a well-oiled **crew**.



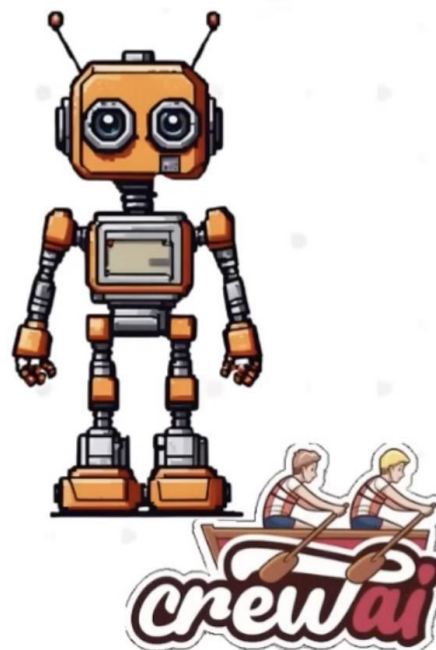
Task: develop a game



Task: blog post



Task: trip planner



Agents

What are CrewAI Agents and how to use them.

What is an agent?

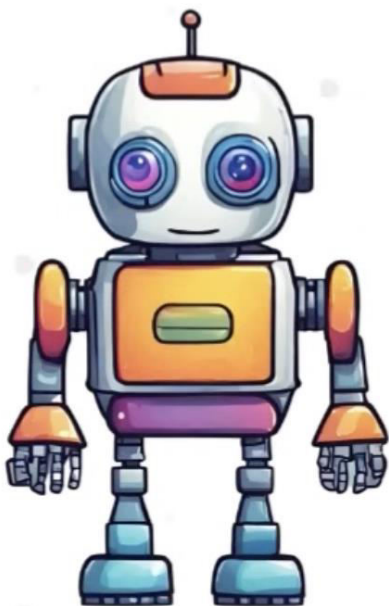
An agent is an **autonomous unit** programmed to:

- Perform tasks
- Make decisions
- Communicate with other agents



Think of an agent as a member of a team, with specific skills and a particular job to do. Agents can have different roles like **Researcher**, **Writer**, or **Customer Support**, each contributing to the overall goal of the crew.

Agents



...and more!

Attribute	Description
role	the role of the agent, what do you want this agent for
goal	the objective of the agent
backstory	the backstory of the agent, gives more context about the role and goal
LLM	you can choose the specific language model to run this agent
function_calling_llm	the language model that will be for tool calling
allow_code_execution	enable code execution for the agent. Default is False.
memory	whether the agent should have memory or not
verbose	gives more output to what the agent is doing behind the scenes in the terminal
tools	the tools this agent can use, such as SearchTools or GitHub Tools

```
from crewai import Agent

agent = Agent(
    role='Data Analyst',
    goal='Extract actionable insights',
    backstory="""You're a data analyst at a large
company.""",
    tools=[my_tool1, my_tool2]
)
```

```
from crewai import Agent
```

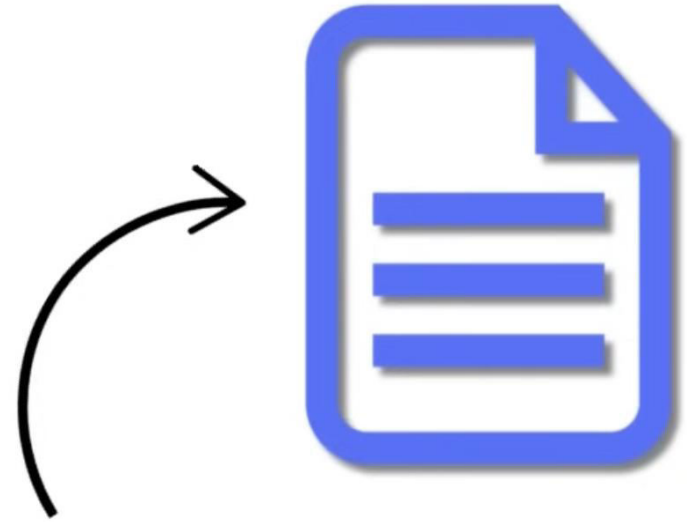
```
@agent
```

```
def poem_writer(self) -> Agent:
```

```
    return Agent(
```

```
        config=self.agents_config['poem_writer'],
```

```
)
```



Tasks

Detailed guide on managing and creating tasks within the CrewAI framework, reflecting the latest codebase updates.

Overview of a Task

In the CrewAI framework, a **Task** is a specific assignment completed by an **Agent**.

They provide all necessary details for execution, such as a description, the agent responsible, required tools, and more, facilitating a wide range of action complexities.

Tasks within CrewAI can be collaborative, requiring multiple agents to work together. This is managed through the task properties and orchestrated by the Crew's process, enhancing teamwork and efficiency.

```
from crewai import Task
```

```
task = Task(  
    description='Find and summarize the latest and  
most relevant news on AI',  
    agent=sales_agent,  
    expected_output='A bullet list summary of the top  
5 most important AI news',  
)
```

```
from crewai import Task
```

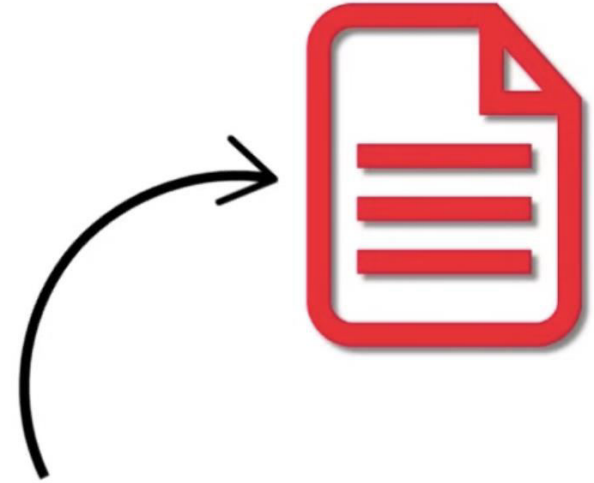
```
@task
```

```
def write_poem(self) -> Task:
```

```
    return Task(
```

```
        config=self.tasks_config['write_poem'],
```

```
)
```



Crew



Attribute	Parameters	Description
tasks	tasks	lists of tasks assigned to the crew
agents	agents	list of agents which are part of the crew
process	process	The process flow that the crew will follow (e.g., sequential)
manager agent	manager_agent	manager sets a custom agent that will be used as a manager.
verbose	verbose	The verbosity level for logging during execution. Defaults to False .
planning	planning	Adds planning ability to the Crew. When activated before each Crew iteration, all Crew data is sent to an AgentPlanner that will plan the tasks and this plan will be added to each task description.

```
@crew
def crew(self) -> Crew:
    """Creates the Research Crew"""
    return Crew(
        agents=self.agents,
        tasks=self.tasks,
        process=Process.sequential,
        verbose=True,
    )

crew_output = crew.kickoff()
```

Coding...

- We will use *cursor* IDE, you can also use pycharm or anyother IDE
 - Install *cursor*
 - install *conda*
- Steps:
 - Create a new project with appropriate name
 - On terminal, setup environment and activate it
 - *conda create -n crewai-project python=3.11.9*
 - *conda activate crewai-latest-dev*
 - On terminal, install crewai and its tools
 - *pip install crewai 'crewai[tools]'*
 - Finally, create a crew,
 - *crewai create crew ai-latest-developments*

Flows

Learn how to create and manage AI workflows using CrewAI Flows.

Introduction

CrewAI Flows is a powerful feature designed to streamline the creation and management of AI workflows. Flows allow developers to combine and coordinate coding tasks and Crews efficiently, providing a robust framework for building sophisticated AI automations.

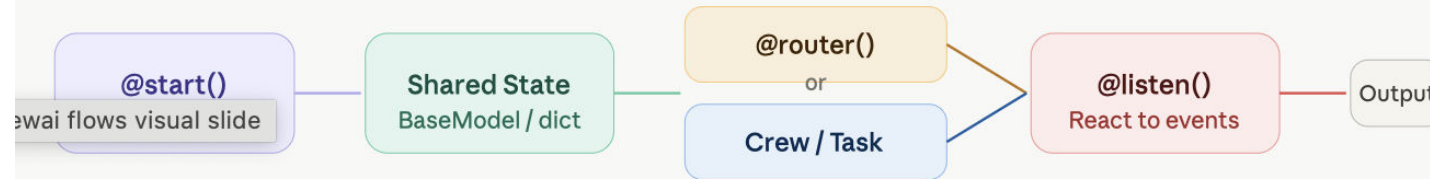
Flows allow you to create structured, event-driven workflows. They provide a seamless way to connect multiple tasks, manage state, and control the flow of execution in your AI applications. With Flows, you can easily design and implement multi-step processes that leverage the full potential of CrewAI's capabilities.

Understanding Flows

crewai >= 0.70

CrewAI Flows is an event-driven orchestration layer that lets you build structured, stateful pipelines connecting AI crews and plain Python tasks — with fine-grained control over execution order and data handoffs.

EXECUTION MODEL



KEY CONCEPTS



@start()

Marks the entry point. Executes first when `flow.kickoff()` is called.



Shared State

A Pydantic `BaseModel` passed between all steps — typed, validated, persistent.



@router()

Returns a string key to branch execution — like a switch on state or output.



@listen()

Subscribes a method to another method's completion event. Enables fan-out.



or_() / and_()

Logical conditions on listeners — trigger when any or all dependencies finish.



kickoff() + plot()

Starts execution and optionally renders an interactive HTML flow graph.

Why Custom Tools?

Not all Tools available can support what we need to do.

This is why CrewAI enables us to create custom tools.

We will create a custom tool to create draft Emails for your Gmail Account.

```
from crewai.tools import BaseTool

class MyCustomTool(BaseTool):
    name: str = "Name of my tool"
    description: str = "Clear description for what this  
tool is useful for, your agent will need this  
information to use it."

    def _run(self, argument: str) -> str:
        return "Result from custom tool"
```



A project of meeting minutes..

1. Create Flow
2. Use Multiple Tools
3. Create Custom Gmail Tool
4. Create a Crew
5. Different Models
6. Integration with Gmail